

CONTENTS

- Overview
- 1 Human memory as the map
- 2 CoALA: naming the four
- 3 Working memory: the context window
- 4 Semantic memory: what the agent knows
- 5 Procedural memory: how the agent does things
- 6 Progressive disclosure
- 7 Episodic memory: what happened before
- 8 Forgetting is an engineering problem
- 9 Not every agent needs all four
- Glossary
- Check yourself
- Where to go next

The Four Types of Agent Memory: Working, Semantic, Procedural, Episodic

Know which store to reach for, what each costs you in context, and why forgetting is the hard part.

For engineers building agents who have to decide what the system remembers, and where · 26 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- What a context window is and that it is finite
- Basic familiarity with agents calling tools
- Comfort reading Python

Overview

A language model is stateless. Every capability that looks like memory is something the surrounding system built, and the useful question is not whether your agent has memory but which kind, held where, loaded when.

The CoALA framework — Cognitive Architectures for Language Agents, from a Princeton research team — names four types, and they map cleanly onto how people remember. Working memory is what is active right now. Semantic memory is what you know. Procedural memory is what you know how to do. Episodic memory is what happened to you before.

The distinction is worth more than a taxonomy, because the four differ in exactly the ways that matter to an implementation: whether they survive the session, whether they cost context tokens continuously or only on demand, who writes to them, and how they go stale. Working memory is volatile and always loaded. Semantic memory is stable and usually always loaded. Procedural memory is loaded on demand. Episodic memory accumulates by itself, and that is the one that turns into an engineering problem.

KEY TAKEAWAYS

- ✓ Models are stateless — every kind of agent memory is machinery the surrounding system provides.
- ✓ CoALA names four types: working, semantic, procedural and episodic, mirroring human memory.
- ✓ Working memory is the context window: fast, always present, volatile, and capped in size.
- ✓ Semantic memory is what the agent knows in general, and in production it is often just Markdown files rather than a vector database.
- ✓ Procedural memory is how the agent does things, packaged as skills with step-by-step instructions.
- ✓ Progressive disclosure keeps procedural memory affordable: advertise a short index, load the full instructions only on a match.
- ✓ Episodic memory is distilled experience, not saved transcripts — the compression is what makes it useful.
- ✓ Forgetting is the hardest part, because staleness and obsolescence have to be decided in code rather than felt.

01 Human memory as the map

0:00

Four kinds of remembering that people do without noticing, which turn out to be four separate engineering problems.

Start with how people remember, because the categories are already familiar and the analogy holds unusually well.

There is short-term memory: what is active in your head right now, this sentence as you read it. There is factual knowledge: your company's security policies, or that Python is an interpreted language. There are learned skills: how to ride a bicycle, how to write backwards on glass. And there is personal experience: the time you spent three hours debugging a Kubernetes cluster before discovering you were pointing at the wrong cluster.

Those four are not variations on one thing. They are retrieved differently, they last for different lengths of time, and they fail differently. You can know a fact and be unable to perform the skill it describes. You can perform a skill without being able to state the facts behind it. You can vividly recall an experience while having forgotten what it taught you.

Well-designed agents need the same four, for the same reason — and the engineering follows the split. Each type has a different storage medium, a different moment at which it enters the model's input, a different writer, and a different way of going wrong. Treating them as one undifferentiated 'memory' feature is how you end up with a system that either forgets everything between sessions or blows its entire context budget on the first turn.

KEY POINTS

- Human memory splits into short-term, factual knowledge, learned skills and personal experience.
- The four are retrieved differently, persist differently and fail differently.
- Knowing a fact and being able to perform a skill are separate capabilities.
- Agents need the same four, and each maps to different machinery.
- Treating memory as one feature produces either amnesia or context exhaustion.

Four kinds of remembering, four kinds of failure

The right-hand column is what makes this a useful taxonomy rather than a tidy one: each type breaks in its own way, so each needs its own fix.

human	agent	fails by
what is on my mind now	working memory	overflowing, or vanishing
facts I know	semantic memory	going out of date
things I can do	procedural memory	never being loaded
what happened to me	episodic memory	piling up and going stale

02 CoALA: naming the four

1:25

A framework from a Princeton research team that maps out the distinct memory types an agent needs.

The framework is CoALA — Cognitive Architectures for Language Agents. It maps out four distinct memory types agents need, and the value of naming them is mostly that it stops separate concerns from being merged into one vague requirement.

The practical way to hold the distinction is to ask four questions of any piece of information your agent has: how long does it last, who writes it, when does it enter the prompt, and how does it become wrong. The answers separate cleanly, and once you have them the implementation follows almost mechanically.

Working memory lasts one session, is written by the runtime, is always in the prompt, and fails by overflowing. Semantic memory lasts indefinitely, is written by humans, is usually loaded at session start, and fails by drifting away from the truth. Procedural memory lasts indefinitely, is written by humans, is loaded on demand, and fails by not being loaded when it was needed. Episodic memory lasts indefinitely, is written by the agent itself, is retrieved by relevance, and fails by accumulating.

That last row is the interesting one. Episodic memory is the only type the agent writes to itself, which is what makes it the only one that can improve without anybody editing a file — and the only one that can quietly fill up with rubbish.

KEY POINTS

- CoALA identifies four memory types for language agents.
- Four questions separate them: lifetime, writer, load moment, failure mode.
- Working memory is written by the runtime; semantic and procedural by humans.
- Episodic memory is the only type the agent writes for itself.
- That self-writing property is both why it can improve and why it degrades.

The four, on the properties that decide the design

Read the columns rather than the rows. 'When it enters the prompt' predicts your token bill; 'who writes it' predicts who is responsible when it is wrong.

	lifetime	written by	enters prompt	fails by
working	session	runtime	always	overflow
semantic	permanent	humans	session start	drift
procedural	permanent	humans	on demand	not being found
episodic	permanent	the agent	by relevance	accumulation

The read path and the write path

Three of the four are read-mostly, maintained by people editing files. Only episodic memory has a write path that runs without a human in it — which is where the design effort goes.

READ (every turn)

```

semantic      ┌
procedural    ┌┴─> working memory ─> model
episodic      ┌
              (context window)

```

WRITE

```
semantic    <— a person edits a file
procedural  <— a person edits a file
episodic    <— the agent decides something is worth keeping
              └─ no human in this loop
```

03 Working memory: the context window

2:09

Everything the agent can see right now — the conversation, the system instructions, whatever has been loaded into the prompt.

Working memory is the context window. It holds the current conversation, the system instructions, and any files or data loaded into the prompt. It is the scratch pad.

The standard analogy is RAM, and it is apt on all three counts. It is fast, because there is no retrieval step — the content is simply there. It is volatile, so when the session ends it is gone. And it is limited in size.

The biggest windows available today run to a million tokens or more, which sounds like enough that the ceiling stops mattering. It does not. Stuff too much in and performance degrades as the model loses track of material buried in the middle — so the usable capacity is smaller than the advertised capacity, and the gap is not a fixed number you can look up.

The implication for design is that context is a budget to be allocated rather than a container to be filled. Every token spent on a document that turns out to be irrelevant is a token not available for the conversation, and it also competes for the model's attention. Which is why the other three memory types exist: they are all, in the end, strategies for keeping things out of working memory until they are needed.

Note also that working memory is not what distinguishes an agent. Every chatbot has a context window. The question that separates the two is what else the system has.

KEY POINTS

- Working memory is the context window: conversation, instructions, loaded data.
- Like RAM — fast because there is no retrieval, volatile, and capped.
- Million-token windows still have a ceiling, and buried content gets lost before it is reached.
- Usable capacity is lower than advertised capacity, by an amount you have to measure.
- Context is a budget to allocate, not a container to fill.
- Every chatbot has working memory; it is not what makes something an agent.

Where the budget actually goes

Worth computing for your own agent before adding anything to the prompt. Teams are routinely surprised by how much is spent before the user has said a word — and the fixed costs are paid on every single turn.

```
def budget(window=200_000):
    fixed = {
        "system prompt": 1_200,
        "tool schemas": 3_400, # grows with every tool you add
        "semantic (CLAUDE.md)": 2_800, # loaded at session start
        "skill index": 900, # ~100 tokens x 9 skills
    }
    spent = sum(fixed.values())
    for name, n in fixed.items():
        print(f" {name:<22} {n:>7,}")
    print(f" {'--> left for work':<22} {window - spent:>7,} "
          f"({(window - spent) / window:.0%})")

# every turn pays the fixed cost again – it is not amortised
```

Volatile means volatile

The single most common misunderstanding about agent memory. Nothing said in a session survives it unless something explicitly wrote it somewhere else — which is precisely the job of the other three types.

```
session 1
user: "we deploy with make ship, never npm run deploy"
agent: "understood"

session 2
user: "deploy it"
agent: runs npm run deploy

# nothing was wrong with the model. the correction lived in working
# memory, the session ended, and no write path existed to keep it.
```

04 Semantic memory: what the agent knows

3:33

The knowledge base — facts, rules, conventions, documentation. In production it is

frequently just Markdown.

Semantic memory is the agent's knowledge base: facts, rules, conventions, documentation. The academic literature describes it in terms of vector databases and knowledge graphs, and those are real implementations.

But in a great many production agentic systems today, semantic memory is something much simpler. It is Markdown files. Claude Code is the example worth studying: a `CLAUDE.md` file sits in the root of a project and holds the project architecture, the coding conventions, the build commands, which frameworks to use, and what not to do. That file is loaded into the context window at the start of every session.

The simplicity is a feature rather than a compromise, and it is worth being explicit about why. A Markdown file is readable, so you can audit exactly what the agent believes. It is diffable, so a change shows up in review. It is version-controlled alongside the code it describes, so it moves with the branch. And it costs one file read instead of an embedding pipeline and a vector store to keep in sync.

The trade is scale. A file is loaded whole, so it competes directly with the context budget, which puts a practical ceiling on it in the low thousands of tokens. Past that you do want retrieval, with the same caveats retrieval always carries. The rule of thumb: if it is small, stable and always relevant, put it in a file; if it is large or only occasionally relevant, retrieve it.

Semantic memory tells the agent what it needs to know in general. Without it, the agent is destined to make the same mistakes repeatedly, because it has no persistent knowledge to draw on.

KEY POINTS

- Semantic memory holds facts, rules, conventions and documentation.
- Vector databases and knowledge graphs are real implementations, and often unnecessary.
- In production it is frequently a Markdown file loaded at session start.
- Files are auditable, diffable and version-controlled with the code they describe.
- Files are loaded whole, so they compete with the context budget and cap out in the low thousands of tokens.
- Small, stable and always relevant → a file. Large or occasionally relevant → retrieval.

What belongs in a semantic memory file

Note how much of it is prohibitions. The mistakes an agent repeats are more expensive than the facts it lacks, and 'what not to do' is the part nobody thinks to write down.

```
# CLAUDE.md

## Architecture
Monorepo. `api/` is FastAPI, `web/` is Next.js, shared types in `packages/`.

## Commands
build:  make build          (never `npm run build` directly)
test:   make test           (runs unit + contract)
deploy: make ship           (staging only; production is manual)

## Conventions
- All timestamps stored UTC, rendered Europe/Madrid.
- Money is integer cents. Never float.
- Migrations are forward-only.

## Do not
- Do not edit `packages/types/generated/` – it is code-generated.
- Do not add dependencies without asking.
- Do not touch `legacy/` unless the task names it.
```

When a file stops being the right answer

The decision is about size and hit rate rather than sophistication. Reaching for a vector store before the file is actually too big buys you an indexing pipeline and a sync obligation in exchange for nothing.

in a file	retrieved
build commands	the full API reference
coding conventions	every past incident report
what not to touch	the whole design-doc archive
the 20 facts always relevant	the 20,000 occasionally relevant

test: would you want this in the prompt on a turn that has
nothing to do with it? if no, it should be retrieved.

05 Procedural memory: how the agent does things

4:59

Skills — a folder with a Markdown file describing what the skill does and the steps to perform it.

Procedural memory is how the agent knows how to do things. There is an open standard for it, agent skills, using a `SKILL.md` file format.

A skill is a folder containing a Markdown file that describes the skill, what it does, and step-by-step instructions for performing it. The scope is wide: creating a PowerPoint presentation, running a structured code review, migrating a database schema, filing a release.

The distinction from semantic memory is worth holding precisely, because it is easy to collapse and expensive to get wrong. Semantic memory is declarative — it states what is true. Procedural memory is imperative — it states what to do, in what order. An agent can have every relevant fact and still perform the steps wrong, and that is a procedural gap that no amount of additional facts will close.

A good test: if the content answers 'what is the case?', it is semantic. If it answers 'how do I do this, and in what order?', it is procedural. Build commands are semantic. The release checklist that uses them is procedural.

The practical properties are what make skills attractive. A skill is a file, so it is reviewable in a pull request. It is portable, so it works with any capable model. It is edited rather than trained, so the loop from noticing a problem to fixing it is minutes. And — the point of the next section — it is not loaded until it is needed.

KEY POINTS

- Procedural memory is instructions for performing a task, packaged as a skill.
- A skill is a folder with a SKILL.md describing what it does and the steps to do it.
- Semantic memory is declarative; procedural memory is imperative.
- An agent can hold every relevant fact and still perform the steps wrong.
- Test: 'what is the case?' is semantic, 'how, in what order?' is procedural.
- Skills are reviewable files, portable across models, and fixed by editing.

A skill folder

The frontmatter is not documentation — the description is the only thing the agent sees until the skill is loaded, so it is what decides whether the skill is ever used at all. The supporting files stay on disk until execution needs them.

```
skills/release/
├─ SKILL.md          what it does + the steps
├─ checklist.md      pulled in during execution
└─ rollback.sh       pulled in only if a step fails

# SKILL.md
---
name: release
description: Cut and publish a release. Use when asked to ship, tag,
  publish or roll back a version.
---

1. Confirm main is green in CI. Stop if not.
2. Bump the version in `pyproject.toml`. Nothing else.
3. Update CHANGELOG.md from commits since the last tag.
4. Tag `v<version>` and push the tag, not the branch.
5. Watch the publish job. On failure run `rollback.sh <version>`.
```

The same subject, split across two memory types

Both are about deployment and they are not interchangeable. Give an agent only the left column and it knows the commands but invents the order; only the right column and it follows steps it cannot ground.

semantic (CLAUDE.md)	procedural (skills/release/SKILL.md)
"deploy: make ship"	"1. confirm CI is green, stop if not"
"staging only"	"2. bump the version, nothing else"
"migrations forward-only"	"3. tag, push the tag not the branch"
facts about the world	an ordered procedure to follow

06 Progressive disclosure

5:41

The agent advertises a lightweight index, loads full instructions on a match, and pulls in

supporting files only during execution.

Skills use progressive disclosure, and without it procedural memory would not be affordable. The agent does not load all its skills into working memory at once, because a large enough set would consume the entire context budget before any work started.

Instead the agent sees a lightweight index — just the name and description of each available skill, perhaps a hundred tokens each. When a task arrives that matches one of those descriptions, the agent loads the full instructions. And if the skill references other material — files, templates, scripts — those are pulled in only when execution actually reaches them.

Three stages, each paying only for what the current task needs: advertise, load, execute.

The arithmetic is what makes the case. Fifty skills at full length might be sixty thousand tokens, which is unusable. The same fifty as index entries are about five thousand, which is a rounding error, and the one skill you actually load costs another twelve hundred. You get the capability of fifty procedures at the context cost of roughly one.

The design consequence is that the description field carries disproportionate weight. It is the only part of the skill the agent can see when deciding whether to reach for it, so a skill with a vague description is effectively invisible — the instructions can be perfect and will never be read. Write the description as a trigger condition, naming the situations and the words a user would actually use.

This is also the sharpest contrast with semantic memory, where the knowledge is always present in context.

KEY POINTS

- Loading every skill at once would exhaust the context budget before work began.
- The agent sees only a name and description per skill — roughly a hundred tokens each.
- Full instructions load when a task matches; referenced files load during execution.
- Fifty skills cost about one skill's worth of context until one is actually used.
- The description is the only visible part at selection time, so a vague one makes the skill invisible.
- Semantic memory is always present; procedural memory is deliberately not.

Three stages, three costs

Run the numbers for your own skill set. The point is that stage one scales with how many skills you have and stages two and three scale with what the current task needs — which is the only reason a large skill library is possible at all.

```
stage 1  ADVERTISE   every turn
  release   : "Cut and publish a release. Use when asked to ship..."
  code-review: "Run a structured review. Use when asked to review..."
  ... 48 more                                     ~100 tokens each
                                              

---


                                              ~5,000 total

stage 2  LOAD        only on a match
  the full text of skills/release/SKILL.md         ~1,200 tokens

stage 3  EXECUTE     only when reached
  checklist.md when step 3 runs                   ~800 tokens
  rollback.sh  only if a step fails                0 tokens

# 50 skills fully loaded would be ~60,000. this is ~7,000.
```

Descriptions are trigger conditions

The instructions inside both skills could be identical. Only the second one will ever be reached, because the first does not contain any of the words a person would use when they need it.

```
# invisible
description: Release management utilities.

# reachable
description: Cut and publish a release. Use when the user asks to
  ship, tag, publish, cut a version, or roll back a bad deploy.

# name the situation and the words the user will actually say
```

07 Episodic memory: what happened before

6:25

The record of past interactions and decisions, and what was learned from them — distilled rather than stored whole.

Episodic memory is the agent's record of past interactions and decisions, and what it learned from them.

The naive implementation is to save every conversation transcript and search them when needed. That technically counts, and it is usually not very useful. A forty-five minute debugging session is mostly wrong turns, restated context and dead ends; searching it returns long passages whose relevance is hard to judge and whose token cost is high.

What production systems do is distillation. As the agent works across sessions it accumulates notes for itself, but it does not save everything — it decides what is worth keeping based on whether the information would actually be useful in a future conversation. The result is compressed experience. 'Last time we debugged the auth module, the issue was in the middleware layer' is far more useful to keep than the transcript of the session that produced it.

The compression is the whole feature. A good episodic note is one sentence, states a durable conclusion rather than a narrative, and would change what the agent does if it were true. Compare that to a transcript, which is long, mostly narrative and mostly no longer relevant.

This is where memory starts to look genuinely like learning, because the agent gets better over time without anyone retraining or editing anything. It is also, for the same reason, the type where things quietly go wrong: nobody reviewed what got written.

KEY POINTS

- Episodic memory records past interactions, decisions and what came of them.
- Saving whole transcripts qualifies technically and works badly.
- Production systems distil: keep the conclusion, discard the narrative.
- A good note is one sentence, durable, and would change future behaviour.
- This is the only type that improves without anyone editing a file.
- It is also the only type nobody reviews before it is written.

Distillation, as a step the agent actually runs

The three conditions are what stop the store filling with restated context. Most sessions should produce nothing — an agent that writes a note every time has not been given a high enough bar.

```
DISTIL = """Review this session. Write at most 3 notes worth keeping.
```

```
Keep a note only if all three hold:
```

- it would change what you do in a future session
- it is still true tomorrow (not "the build is broken right now")
- it is not already in CLAUDE.md or a skill

```
One sentence each. State the conclusion, not the story.
```

```
Return an empty list if nothing qualifies."""
```

```
def close_session(transcript, store):  
    for note in model.extract(DISTIL, transcript, schema=Note):  
        store.write(note.text, tags=note.tags, ts=now())
```

```
# most sessions should produce zero notes
```

What survives, and what should not

The left column is what a transcript search would surface; the right is what the same session is actually worth. Roughly two orders of magnitude smaller, and considerably more useful.

the session	the note worth keeping
45 min of auth debugging 8 wrong hypotheses 3 stack traces a fix in middleware/	"Auth failures usually originate in the middleware layer, not the token parser."
"the build is red"	nothing — not true tomorrow
"user prefers pytest"	"This project uses pytest, not unittest." → actually semantic, put it in CLAUDE.md instead

Retrieval has to be cheap and bounded

Episodic memory is only worth having if reading it costs less than the mistakes it prevents. Retrieve a handful of notes by relevance, never the whole store — and cap it, because the store grows without limit and the context window does not.

```
def recall(task, store, k=5, max_tokens=600):
    hits = store.search(embed(task), k=k)

    out, spent = [], 0
    for note in hits:
        cost = count_tokens(note.text)
        if spent + cost > max_tokens:
            break
        out.append(note)
        spent += cost
    return out

# a hard token cap, not just a result count: notes vary in length
# and the store only ever gets bigger
```

08 Forgetting is an engineering problem

7:52

What do you delete? When does something become obsolete? People forget effortlessly; agents have to be told.

Episodic memory is the hardest of the four to get right, and the difficulty is all on the deletion side.

The questions have no default answers. What do you delete? When does information become obsolete? If a user changes jobs, do you keep the old project memories or drop them? Humans are good at forgetting — frustrating as that is, it is doing real work, discarding what stopped mattering without anyone deciding to.

For agents, forgetting has to be built. And the failure it prevents is worse than a full disk: a stale note is not neutral, it is confidently wrong guidance that the agent will act on. A note saying the deploy command is `npm run deploy`, written before the project moved to `make ship`, is more damaging than no note at all, because it displaces the correct behaviour rather than merely failing to supply it.

Three mechanisms cover most of it, and they compose. Time-based decay handles notes whose relevance fades. Contradiction handling replaces a note when a newer one disagrees, rather than keeping both and letting retrieval pick. Scoped invalidation drops a whole class of notes when the context they belonged to ends — a project archived, a job changed, a service decommissioned.

And one habit is worth more than any of them: make the store readable. A note file a person can open, skim and delete lines from turns an intractable retention policy into a five-minute review. The systems that stay useful are the ones where somebody occasionally looks.

KEY POINTS

- Deletion, not writing, is the hard part of episodic memory.
- A stale note is worse than a missing one — it displaces correct behaviour.
- Time-based decay handles relevance that fades.
- Contradiction handling replaces superseded notes rather than storing both.
- Scoped invalidation drops a class of notes when its context ends.
- A human-readable store makes retention a five-minute review instead of a policy.

Three mechanisms that compose

None is sufficient alone. Decay misses notes that are wrong immediately; contradiction handling misses notes nothing contradicts; scoped invalidation misses everything not tied to a scope.

```
def maintain(store, now):
    # 1. decay – relevance fades, and unused notes fade fastest
    for note in store.all():
        age_days = (now - note.last_used).days
        if age_days > 90 and note.use_count == 0:
            store.drop(note)

    # 2. contradiction – replace, never keep both
    for note in store.all():
        newer = store.contradicting(note, after=note.ts)
        if newer:
            store.drop(note)          # retrieval must not have to choose

    # 3. scoped invalidation – the context itself ended
    for scope in store.dead_scopes(): # archived repo, closed account
        store.drop_where(scope=scope)
```

Why a stale note beats no note, in the wrong direction

The empty store produces a question. The stale store produces a confident wrong action. This asymmetry is the argument for aggressive deletion — when in doubt, drop it.

```
no memory
  user: "deploy it"
  agent: "How do you deploy this project?"      ← mildly annoying

stale memory
  note: "deploy with npm run deploy"    (written 6 months ago)
  user: "deploy it"
  agent: runs npm run deploy              ← wrong, and confident

# the note did not fail to help. it actively replaced the right answer.
```

Keep the store legible

The most effective retention policy in practice is a file somebody can read. Opaque embeddings in a vector store are never audited, so nothing is ever deleted, so the store degrades — the fix is structural, not algorithmic.

```
memory/
├─ project.md      "Auth failures originate in middleware, not the
│                  token parser." (2026-03-11, used 4x)
├─ prefs.md        "Prefers minimal diffs over refactors."
└─ archive/        moved here rather than deleted, so a wrong
                   eviction is recoverable

# if you cannot read it, you will never delete from it
```

09 Not every agent needs all four

8:32

Match the memory architecture to what the agent actually does. Most need fewer types than the taxonomy suggests.

Four types exist; your agent probably does not need all of them, and adding one you do not need costs context, latency and a maintenance surface.

A simple reflex agent — a thermostat, a basic routing bot — needs working memory and nothing else. It maps an input to an action with no notion of before or after.

A narrow customer support agent, say one that resets passwords, needs working memory plus procedural memory: it has to recall the password-reset skill and follow the steps. It does not need to remember individual users across sessions, and giving it episodic memory would create a retention problem in exchange for nothing.

A coding agent probably needs all four. Working memory for the current task. Semantic memory for the project's architecture and conventions. Procedural memory for its skill system. And episodic memory to learn across sessions.

Work upward from working memory and add a type only when you can name the failure it fixes. 'The agent keeps forgetting our conventions' is semantic. 'It knows the facts but does the steps in the wrong order' is procedural. 'It makes the same discovery every session' is episodic. If you cannot name the failure, you do not need the type yet.

What all this adds up to is that memory is what separates a chatbot from an agent. A chatbot gives a response. An agent gives a response shaped by persistent knowledge and accumulated experience — it remembers the project, it remembers preferences, and a good memory architecture remembers the mistakes, so they do not have to be repeated.

KEY POINTS

- A reflex agent needs working memory only.
- A narrow task agent needs working plus procedural.
- A coding agent typically needs all four.
- Add a memory type only when you can name the failure it fixes.
- Unneeded memory costs context, latency and maintenance.
- Memory is what separates a chatbot's response from an agent's.

Three agents, three architectures

Notice the middle row. Skipping episodic memory there is a decision, not an omission — a password-reset agent that remembers users has taken on a data-retention problem for no gain.

	working	semantic	procedural	episodic
thermostat / router	●	·	·	·
password-reset agent	●	·	●	·
coding agent	●	●	●	●

Name the failure before adding the type

Each symptom points at exactly one type, and the fixes are not interchangeable. Adding episodic memory to solve a semantic problem produces notes that restate what should have been a file.

symptom	add
"forgets our conventions every session"	semantic
"knows the facts, botches the order"	procedural
"rediscovers the same thing every week"	episodic
"loses track halfway through long tasks"	none – budget working memory better

cannot name the symptom? you do not need the type yet.

The routing question, per turn

Once the types are distinct, the runtime decision is small. Semantic is already loaded, procedural is matched against the index, episodic is retrieved by relevance and capped — and most turns touch only the first.

```
def assemble_context(task, session):
    parts = [SYSTEM, session.semantic]          # always present

    skill = match_skill(task, SKILL_INDEX)      # on demand
    if skill:
        parts.append(skill.load())

    notes = recall(task, EPISODIC, k=5, max_tokens=600)  # by relevance
    if notes:
        parts.append(format_notes(notes))

    return parts + session.messages             # working memory
```

Glossary

CoALA

Cognitive Architectures for Language Agents, a framework from a Princeton research team that identifies four memory types agents need.

Working memory

The context window: the conversation, instructions and loaded data the agent can see right now. Fast, volatile and capped.

Semantic memory

What the agent knows in general — facts, rules, conventions and documentation. Often a Markdown file loaded at session start.

Procedural memory

How the agent performs a task: ordered, step-by-step instructions, packaged as a skill.

Episodic memory

The record of past sessions and what was learned from them, kept as distilled notes rather than whole transcripts.

Context window

The maximum text a model can process in one call. It bounds working memory and everything loaded into it.

Progressive disclosure

Advertising skills by a short description, loading full instructions only on a match, and supporting files only during execution.

Skill index

The lightweight list of skill names and descriptions the agent always sees — roughly a hundred tokens per skill.

SKILL.md

The Markdown file defining a skill: what it does, when to use it, and the steps to perform it.

CLAUDE.md

A project-root Markdown file holding architecture, conventions, commands and prohibitions, loaded at the start of every session.

Distillation

Compressing a session into a few durable conclusions worth remembering, instead of storing the transcript.

Scoped invalidation

Dropping a whole class of memories when the context they belonged to ends — a project archived, an account closed.

Reflex agent

An agent that maps an input directly to an action with no persistence, needing only working memory.

Stale note

A stored memory that has become false. Worse than no memory, because it displaces correct behaviour rather than merely omitting it.

Check yourself

Answer first, then open each one.

An agent is told a project's deploy command mid-session and follows it correctly, then gets it wrong the next day. Which memory type is missing, and why is this not a model failure?

Semantic memory. The correction lived in working memory, which is volatile and ends with the session, and no write path existed to persist it. The model behaved correctly on both days given what it could see — the missing piece is a file loaded at session start, not a better model.

Why does progressive disclosure make a library of fifty skills affordable when loading them all would not be?

Because the always-present cost is only the index — a name and description per skill, roughly a hundred tokens each — while full instructions load only when a task matches and supporting files only when execution reaches them. Fifty skills cost around five thousand tokens instead of sixty thousand, so the context cost scales with what the current task needs rather than with how many skills exist.

A skill has perfect instructions and is never used. What is the most likely cause?

Its description. That is the only part visible at selection time, so if it does not name the situation and the words a user would actually say, the agent has no basis for loading it. The instructions are never read because the decision to read them happens before they are visible.

Why do production systems distil episodic memory rather than saving conversation transcripts?

A transcript is mostly wrong turns, restated context and dead ends, so retrieving it returns long passages of doubtful relevance at high token cost. A distilled note keeps the durable conclusion and discards the narrative, which is both far smaller and far more likely to change what the agent does next.

Why is a stale episodic note worse than having no episodic memory at all?

An empty store makes the agent ask, which is mildly inconvenient. A stale note makes it act confidently on information that has become false, displacing the correct behaviour rather than merely failing to supply it. This asymmetry is the argument for deleting aggressively when in doubt.

A password-reset support agent is given episodic memory so it can recall individual users. What has that decision actually bought and cost?

It has bought nothing the task needs — each reset is self-contained and the procedure comes from procedural memory. It has cost a data-retention obligation, a staleness problem and per-turn retrieval latency. Memory types should be added only when you can name the failure they fix.

Where to go next

- Compute the fixed context cost of your own agent — system prompt, tool schemas, semantic file, skill index — and see what fraction of the window is gone before the user has typed anything.
- Read the CoALA paper (Sumers, Yao, Narasimhan and Griffiths, 2023) for the fuller treatment, including how the memory types connect to the agent's decision procedure.
- Take an agent you already run and classify everything currently in its system prompt as semantic, procedural or neither; the procedural parts usually belong in skills that are not always loaded.
- Write the distillation step for a session you actually had, then check a week later how many of the notes are still true — the survival rate tells you whether your bar is high enough.

- Instrument episodic retrieval to record which notes were retrieved and whether they were used, then delete anything never used after 90 days and see if anything breaks.
 - Rewrite the descriptions of skills that never trigger as explicit trigger conditions naming user phrasings, and measure how often they are selected afterwards.
 - Try the reverse experiment: strip semantic memory from a coding agent for a day and note how many corrections you repeat — that count is what the file is worth.
-

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.